

How to secure application code

The first obvious challenge is to find security weaknesses in the code. Security code reviews before each version release have been followed for a very long time, but they come very late in the cycle and need time from specialists. Static code analysis tools can help overcome the problem.

Static application security testing (SAST) can scan the code for bad security practices and identify vulnerabilities. But how does one choose a good SAST tool? The choice will depend on:

- Support needed for the programming languages used in the organisation
- Number of rules or tests
- Whether it covers all the OWASP Top 10 vulnerabilities
- Whether it covers all the common weakness enumerations (CWEs)
- Suggests remediations
- Has the option to suppress false positives (not the entire rule, only the specific instance)
- Has options to add custom detection rules

Some popular tools for static code analysis are given in Figure 4.

Tip: Use tools focused on your programming language. For example, *gosec* finds more vulnerabilities compared to paid tools.

One more thing! How about secrets like API keys, auth tokens, encryption keys, etc? We all know that secrets should not be committed to the repositories. But accidents can happen. To solve this problem, we can follow two approaches.

Preventive: Have a pre-commit or pre-push hook to check whether any secrets are present in the commit, and block them from getting created.

Reactive: Run a secrets scanner in the pipeline to detect any secrets that have checked into the repository — before they get leaked.



Figure 3: Intro | Secured pipeline

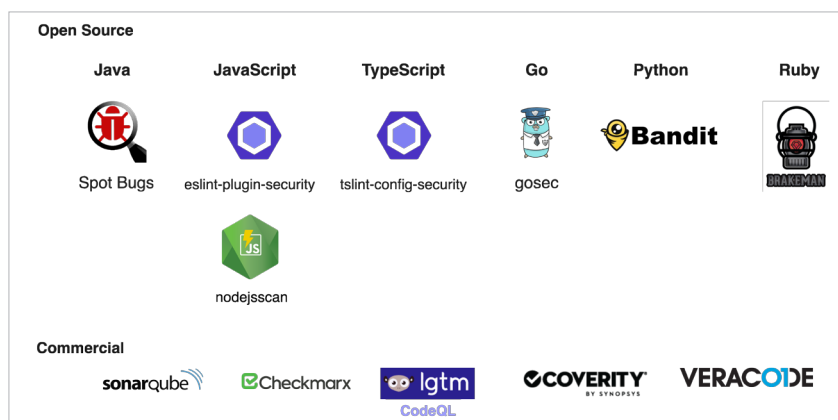


Figure 4: Tools | Static code analysis

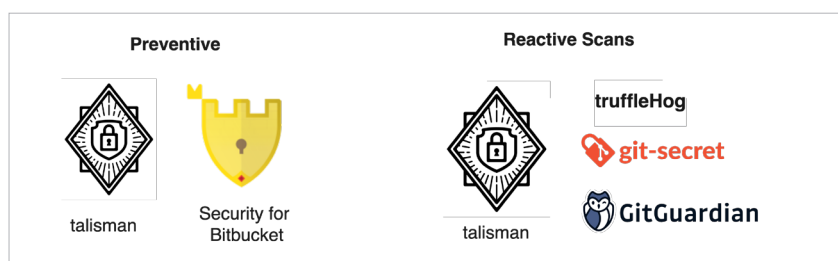


Figure 5: Tools | Secret detection

Is the preventive approach enough on its own? Unfortunately, no. There can be occasions where we fail to run the pre-checks. As an example, a developer who is not part of your team may commit code to your repo. However, you may have switched to a new laptop in which you have not installed all the tools. And you may deliberately skip pre-checks to push the changes through faster. So we need to use both preventive and reactive approaches.

The next obvious question is: What do you do in case a secret is detected in the scan and leaked in public? Well, you should rotate the secrets and follow a proper security incident response plan.

How about application dependencies

A software application depends on a lot of libraries or packages. For example, if an application needs to make HTTP calls, we might use an HTTP library instead of writing